

Nombre: *Jair Patiño*



Sumario

- Introducción a deep learning
- Redes neuronales convolucionales
- Trabajar con pocos datos
- Visualización de CNN

Tensores

- Dato multidimensional (escalar, vector, matriz, tensor)





- Deep learning consiste en una serie de transformaciones (operaciones matemáticas con tensores) del input para dar output.



- Operacion mas importante: tensor dot (matmul)

In [1]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras import layers

model = Sequential()
model.add(layers.Input(shape=(256,)))       # input 2D tensor con dimensiones (256,
model.add(layers.Dense(64))                 # output 1D tensor con dimensión (64)
model.add(layers.Dense(32))                 # (input asumido de la capa anterior) o
```

```
Using TensorFlow backend.
```

In [3]:
```
# mostrar estructura del modelo
model.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_1 (Dense)              (None, 64)                16448
_________________________________________________________________
dense_2 (Dense)              (None, 32)                2080
=================================================================
Total params: 18,528
Trainable params: 18,528
Non-trainable params: 0
_________________________________________________________________
```

Construir redes

- Capas compatibles en las dimensiones output - input

Tipos de capas

- Cada tipo de capa suele tener un uso (dependiente de la red a diseñar según el tipo de entrada y la aplicación):
- **Dense** para tratar input vector o **datos estructurados** (2D tensor de samples,features)
- **LSTM** (Recurrent Neural Networks) para tratar **secuencias lógicas** (3D tensores de samples,timestep,features)
- **Convolucionales** (Convolutional Neural Network) para tratar **imagenes** (4D tensores de samples,height,weight,channels)

Estructura de la red neuronal

- La mas habitual es secuencial (feedforward), pero hay más
- Two-branch, Multihead, Inception
- La arquitectura ideal es un arte: Documentación tipo de problema, experiencia, experimentación.

Deep learning en *Computer Vision*

- Redes convolucionales (CNN)
- Descripción
- Entrenamiento
- Trabajar con pocos datos
- Data augmentation
- Redes pre-entrenadas: Transfer Learning & Fine-tuning
- Visualizando lo que aprenden las CNNs

Convolución

- Dense layers aprenden patrones globales (en toda la imagen)
- Convoluciones detectan patrones locales (resistente a traslaciones)

- Convoluciones permiten aprender jerarquías de patrones (patrones locales como lineas, curvas, etc. a circulos, rectangulos, a constelaciones)

 No description has been provided for this image

Fuente: https://cs.stackexchange.com/questions/16545/what-is-the-difference-between-a-neural-network-a-deep-learning-system-and-a-de

Cómo lo hacen

In [2]:
```
# Fuente: Shafkat, 2018. Acceso el 4 de Enero 2019. https://towardsdatascience.com/
from IPython.display import Image
Image(filename="resources/conv_normal.gif")
```

Out[2]:
```
<IPython.core.display.Image object>
```

- Divide el input (3D tensor) en parches y aplica la operación convolución (la misma en cada capa) a cada parche

- Output (3D tensor) es un mapa de features o **mapa de activaciones** (cada una el resultado de aplicar la transformacion). Dicho mapa tendra dimensiones **Height x Width x Nº filtros**. Cada capa del volumen (eje z) es el resultado de aplicar un filtro a cada parche del input

- La convolución desliza cada parche sobre el input, parando en cada posible posicion y aplicando la transformacion (función kernel)

In [3]:
```
# Fuente: Shafkat, 2018. Acceso el 4 de Enero 2019. https://towardsdatascience.com/
Image(filename="resources/conv_multichannel.gif")
```

Out[3]:
```
<IPython.core.display.Image object>
```

In [4]:
```
# Fuente: Shafkat, 2018. Acceso el 4 de Enero 2019. https://towardsdatascience.com/
Image(filename="resources/conv_multichannel_2.gif")
```

Out[4]:
```
<IPython.core.display.Image object>
```

In [5]:
```
# Fuente: Shafkat, 2018. Acceso el 4 de Enero 2019. https://towardsdatascience.com/
Image(filename="resources/bias_cnn.gif")
```

Out[5]: <IPython.core.display.Image object>

Padding y striding

- Sólo hay 9 formas de aplicar un parche 3x3 a un imagen de 5x5 sin OOB!
- Si se quiere aplicar a todos los pixeles --> **padding** (añadir filas y columnas para hacerlo posible)

`Conv2D(...,padding=<valor>): 'valid' sin padding, 'same' para igualar tamaño input y output feature`

In [6]:
```
# Fuente: Shafkat, 2018. Acceso el 4 de Enero 2019. https://towardsdatascience.com/
Image(filename="resources/conv_padding.gif")
```

Out[6]: <IPython.core.display.Image object>

- **Stride** determina el salto entre ventana y ventana
- Poco habitual su utilización como efecto downsample

`Conv2D(...,strides=<valor>): int, salto entre ventanas`

In [7]:
```
# Fuente: Shafkat, 2018. Acceso el 4 de Enero 2019. https://towardsdatascience.com/
Image(filename="resources/conv_stride.gif")
```

Out[7]: <IPython.core.display.Image object>

Pooling

- Se usan para hacer **downsampling del input feature** (reducir dimensionalidad espacial)
- Similar a convoluciones, pero no aplican un kernel sino que aplican **funcion 'max' (o average)**
- Por defecto, tamaño 2x2 y stride 2

¿Para qué?

- Reducir el numero de parámetros
- Permite a ventanas posteriores aceptar input de areas más grandes en capas anteriores (mejora percepción)

In [2]:
```
# red sin pooling

from tensorflow.keras import models
from tensorflow.keras import layers

convnet_nopooling = models.Sequential()
#BASE MODEL (Destinada a FE)
convnet_nopooling.add(layers.Input(shape=(32,32,3)))
```

```
convnet_nopooling.add(layers.Conv2D(32,(3,3),activation='relu'))
convnet_nopooling.add(layers.Conv2D(32,(3,3),activation='relu'))
convnet_nopooling.add(layers.Conv2D(64,(3,3),activation='relu'))
convnet_nopooling.add(layers.Conv2D(64,(3,3),activation='relu'))
#TOP MODEL
convnet_nopooling.add(layers.Flatten())
convnet_nopooling.add(layers.Dense(512,activation='relu'))
convnet_nopooling.add(layers.Dense(10,activation='softmax'))

convnet_nopooling.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_4 (Conv2D)            (None, 30, 30, 32)        896
_________________________________________________________________
conv2d_5 (Conv2D)            (None, 28, 28, 32)        9248
_________________________________________________________________
conv2d_6 (Conv2D)            (None, 26, 26, 64)        18496
_________________________________________________________________
conv2d_7 (Conv2D)            (None, 24, 24, 64)        36928
_________________________________________________________________
flatten_1 (Flatten)          (None, 36864)             0
_________________________________________________________________
dense_2 (Dense)              (None, 512)               18874880
_________________________________________________________________
dense_3 (Dense)              (None, 10)                5130
=================================================================
Total params: 18,945,578
Trainable params: 18,945,578
Non-trainable params: 0
_________________________________________________________________
```

In [9]:
```
# Red con pooling
convnet_pooling = models.Sequential()

############################ BASE MODEL ###### ENCARGADO DE EXTRAER CARACTERÍSTICAS A
#BLOQUE CONVOLUCIONAL 1
convnet_pooling.add(layers.Input(shape=(32,32,3)))
convnet_pooling.add(layers.Conv2D(32,(3,3),activation='relu'))
convnet_pooling.add(layers.Conv2D(32,(3,3),activation='relu'))
convnet_pooling.add(layers.MaxPooling2D((2,2)))

#BLOQUE CONVOLUCIONAL 2
convnet_pooling.add(layers.Conv2D(64,(3,3),activation='relu'))
convnet_pooling.add(layers.Conv2D(64,(3,3),activation='relu'))
convnet_pooling.add(layers.MaxPooling2D((2,2)))

############################ TOP MODEL ###### ENCARGADO LLEVAR A CABO LA CLASIFICACIÓ
convnet_pooling.add(layers.Flatten())
convnet_pooling.add(layers.Dense(512,activation='relu'))
convnet_pooling.add(layers.Dense(10,activation='softmax'))

convnet_pooling.summary()
```

```
Model: "sequential_2"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_5 (Conv2D)            (None, 30, 30, 32)        896
_________________________________________________________________
conv2d_6 (Conv2D)            (None, 28, 28, 32)        9248
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 14, 14, 32)        0
_________________________________________________________________
conv2d_7 (Conv2D)            (None, 12, 12, 64)        18496
_________________________________________________________________
conv2d_8 (Conv2D)            (None, 10, 10, 64)        36928
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 5, 5, 64)          0
_________________________________________________________________
flatten_2 (Flatten)          (None, 1600)              0
_________________________________________________________________
dense_3 (Dense)              (None, 512)               819712
_________________________________________________________________
dense_4 (Dense)              (None, 10)                5130
=================================================================
Total params: 890,410
Trainable params: 890,410
Non-trainable params: 0
_________________________________________________________________
```

Hiperparámetros de CNN

- **Tamaño de los parches** (habitualmente 3x3 o 5x5)
- Depth (profundidad, **número de filtros**) del mapa de features del output (32 y 64 en el ejemplo)

```
Conv2D(output_depth,(patch_height,patch_width))
```

NOTEBOOK: INTRODUCCIÓN A LAS CONVOLUTIONAL NEURAL NETWORKS: MNIST DATASET

NOTEBOOK: ¿POR QUE CONVOLUTIONAL NEURAL NETWORKS?: CIFAR DATASET

Entrenando CNNs

- Entrenar con pocos datos a la entrada
- Data augmentation para ayudar a paliar el overfitting

Data augmentation

- Incrementar el número de ejemplos para reducir overfitting (junto con dropout)
- Generar datos a partir de los presentes, a través de transformaciones geométricas y de intensidad:
- Escalado, zoom
- Traslaciones, rotaciones, espejo
- Transformaciones de color, *histogram equalization*
- Para implementarlo en la práctica se hace uso de ImageDataGenerators
   No description has been provided for this image

NOTEBOOK: REDUCIENDO OVERFITTING MEDIANTE DATA AUGMENTATION

Utilizando una red pre-entrenada

- Aunque no hayan sido entrenadas con las mismas clases target, han aprendido a distinguir features
- Puede reutilizarse en otra red
- Bueno cuando se tienen pocos datos

Transfer Learning

- Red entrenada en ImageNet: animales, objetos, etc. 1.4 millones
   No description has been provided for this image
- Utilizar la parte convolucional (**base model**) de una red entrenada en un gran dataset para la extracción de características
- La parte densa (**top model**) suele ser más especifica a las clases objetivo -> Normalmente se diseña ad-hoc  No description has been provided for this image

- Dataset del **dominio destino parecido** a dataset del **dominio original** se puede hacer uso de **características de alto nivel**
- Características de bajo nivel son genéricas a problemas de *computer vision*
- Fijar o **congelar** el base model y entrenar solo la parte densa

Fine Tuning

- Dataset del **dominio destino POCO parecido** a dataset del **dominio original** se debe **reentrenar o descongelar PARTE del base model**. Tan solo se puede dejar congelados los filtros que obtienen **características de bajo nivel**
- ¿**Estrategia** para descongelar capas? -> **Desde último bloque convolucional** voy descongelando **uno a uno** (en cada experimento) y viendo que topología ofrece los mejores resultados  No description has been provided for this image

NOTEBOOK: TRABAJANDO CON REDES PRE-ENTRENADAS: TRANSFER LEARNING & FINE-TUNING

Visualizar CNN

- Como las features que aprenden las CNN son representaciones visuales, se pueden visualizar
- Tres formas
- Visualizar activaciones intermedias
- Visualizar filtros convolucionales
- Visualizar heatmaps de las activaciones por clase

Visualizar Activaciones intermedias

- Visualizar el output de las capas de la red
- 2D imagen por canal

Interpretación

- Capas iniciales son como detectores de bordes
- Capas más profundas son más difíciles de interpretar (abstractas) y tienen información relativa a la clase de imagen
- La activación de capas profundas es más dispersa (sparse)

Visualizando filtros convolucionales

- Visualizar las imágenes que maximizan la respuesta a un filtro

Visualización de heatmaps de activacion por clase

- Útil para averiguar qué partes de la imagen contribuyen más a la decisión
- Interesante para saber qué pasa cuando se cometen errores

GITHUB

- https://github.com/Failedbanjo/machine_learning_2